PRISM: Proposed Solution for Privacy-Constrained Customer Churn Classification

Abhishek Majumdar
University of Victoria
Victoria, Canada

Dev Parsana
University of Victoria
Victoria, Canada

Ravi Vashi
University of Victoria
Victoria, Canada

Abstract

PRISM measures how much churn-prediction performance is lost when a model may only see a restricted set of customer attributes. We define three nested information tiers over the IBM Telco data and, for every tier, train a k-nearest-neighbour (k-NN) baseline and a logistic regression (LR) classifier trained with absolute-error loss under an identical protocol: one cleaning step, one stratified hold-out split, one set of stratified cross-validation folds, and model selection by churn F-measure. This report formally defines the input, output and algorithm of the proposed method, gives its pseudo-code, and describes a reproducible implementation.[1]

Keywords

customer churn, data minimization, privacy tiers, k-NN, logistic regression

1 Introduction

Churn models are usually trained on everything a provider knows about its customers [4, 5], while data-minimization work asks how much input a model actually needs [2]. PRISM connects the two by fixing the learning protocol and varying only the information a model may access. Instead of selecting features by how predictive they are [6], each tier is fixed *before* training, so a model never sees an attribute outside its tier. We use k-NN [1] as the baseline and compare LR against it; both were covered in the course, which keeps every modelling decision explainable.

2 Formal Definition

2.1 Input and Output

Let $O = \{o_i\}_{i=1}^{N}$ be the customers (N = 7,043 [3]), each with a churn label $y_i \in \{0, 1\}$ (1 = churned, 26.5% of customers). Let $\mathcal{A}$ be the 19 eligible account attributes; identifiers, location fields and outcome-derived fields (Churn Score, CLTV, Churn Reason) are excluded from $\mathcal{A}$. We define three nested tiers

$$\mathcal{A}_1 \subset \mathcal{A}_2 \subset \mathcal{A}_3 = \mathcal{A}, \qquad |\mathcal{A}_1| = 4,\ |\mathcal{A}_2| = 6,\ |\mathcal{A}_3| = 19,$$

where $\mathcal{A}_1$ holds administrative terms (tenure, contract, payment method, paperless billing), $\mathcal{A}_2$ adds monthly and total charges, and $\mathcal{A}_3$ adds demographics and subscribed services. $\pi_t(o)$ denotes the projection of customer o onto $\mathcal{A}_t$.

Given the labelled data $\mathcal{D} = \{(o_i, y_i)\}$, tiers $\{\mathcal{A}_t\}_{t=1}^{3}$ and models $m \in \{\textsc{knn}, \textsc{lr}\}$, PRISM is a function

$$F(\mathcal{D}) = \big\{(m, t, \theta^*_{m,t}, \mathbf{q}_{m,t})\big\},$$
$$\mathbf{q}_{m,t} = (P, R, F_1, \mathrm{AUC}_{\mathrm{ROC}}, \mathrm{AUC}_{\mathrm{PR}})$$

that returns, for every model and tier, the selected hyperparameters $\theta^*_{m,t}$ and the hold-out quality $\mathbf{q}_{m,t}$ of a classifier $f_{m,t} : \pi_t(o) \mapsto \{0, 1\}$. The key constraint is that $f_{m,t}$ is learned and applied using $\pi_t(o)$ only.

2.2 Representation

Cleaning. Total Charges is stored as text; its 11 blank values all belong to customers with zero tenure, so they are set to 0 (nothing billed yet). Duplicate customers and rows with missing or out-of-domain values are removed (none were found), leaving N = 7,043.

Split. $\mathcal{D}$ is split 80/20, stratified on y, into $\mathcal{D}_{\mathrm{tr}}$ (5,634) and $\mathcal{D}_{\mathrm{te}}$ (1,409, 374 churners). $\mathcal{D}_{\mathrm{tr}}$ is further split into five stratified folds $\mathcal{F}_1, \ldots, \mathcal{F}_5$. One seed ($s$ = 2026) fixes the split, the folds and all LR randomness, so every model and tier sees exactly the same rows.

Encoding. $\phi_t : \pi_t(o) \to \mathbb{R}^{d_t}$ z-scores each numeric attribute a as $(v - \mu_a)/\sigma_a$ and one-hot encodes each categorical attribute with value set V_a (one indicator if $|V_a| = 2$), giving $d_1 = 9$, $d_2 = 11$, $d_3 = 40$. μ_a, σ_a and V_a are always estimated on the training rows of the current fit, so no statistic leaks from validation or test rows. z-scoring is needed because both models use the raw geometry of ϕ_t: otherwise Total Charges (up to $\approx$ 8,700) would dominate distances and gradients.

2.3 Baseline: k-NN

For $\mathbf{x} = \phi_t(\pi_t(o))$, let $N_k(\mathbf{x})$ be the k training customers with the smallest Euclidean distance $\|\mathbf{x} - \mathbf{x}_j\|_2$. The churn score and prediction are

$$s(\mathbf{x}) = \tfrac{1}{k} \textstyle\sum_{j \in N_k(\mathbf{x})} y_j, \qquad f(\mathbf{x}) = \mathbb{1}[s(\mathbf{x}) > 0.5].$$

k is chosen from $\{1, \ldots, 20\}$; if the best k is the largest tested, the range is extended in steps of 10 up to 100, and ties go to the smaller k. k-NN makes no assumption about the shape of the decision boundary and has a single hyperparameter, so it is a transparent baseline.

2.4 Compared Method: Logistic Regression with Absolute-Error Loss

LR scores a customer with $s(\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$ and predicts churn when $s(\mathbf{x}) \geq 0.5$, i.e., $f(\mathbf{x}) = \mathbb{1}[s(\mathbf{x}) \geq 0.5]$. The parameters $\boldsymbol{\theta} = (\mathbf{w}, b)$ minimise the absolute error from the course notebook

$$E(y, \hat{y}) = |y - \hat{y}| = \begin{cases} 1 - \hat{y}, & y = 1 \\ \hat{y}, & y = 0 \end{cases}, \qquad \frac{\partial E}{\partial z} = (1 - 2y)\, \hat{y}(1 - \hat{y}),$$

with $z = \mathbf{w}^\top \mathbf{x} + b$, by mini-batch gradient descent: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \frac{\eta}{|B|} \sum_{i \in B} (1 - 2y_i) \hat{y}_i (1 - \hat{y}_i) [\mathbf{x}_i; 1]$. Weights start in $U(-0.01, 0.01)$ and the rows are reshuffled every epoch. Because $E \in [0, 1]$, a confidently misclassified customer cannot dominate an update; the trade-off is a small gradient for such customers. LR adds interpretability:

[1] Code: https://github.com/ParsanaDev/PRISM-churn-privacy/tree/milestone3-solution

Algorithm 1 PRISM: tiered training and evaluation

Input: data $\mathcal{D}$, tiers $\{\mathcal{A}_t\}$, models $\mathcal{M}$, grids $\{\Theta_m\}$, seed s
Output: $\{(m, t, \theta^*_{m,t}, \mathbf{q}_{m,t})\}$

$\mathcal{D} \leftarrow \textsc{Clean}(\mathcal{D})$
$(\mathcal{D}_{\text{tr}}, \mathcal{D}_{\text{te}}) \leftarrow \textsc{StratifiedSplit}(\mathcal{D}, 0.2, s)$
$\mathcal{F}_1..\mathcal{F}_5 \leftarrow \textsc{StratifiedFolds}(\mathcal{D}_{\text{tr}}, 5, s)$
for $t = 1, 2, 3$ **and** $m \in \mathcal{M}$ **do**
  for $\theta \in \Theta_m$ **do**
    for $j = 1..5$ **do**
      $\phi \leftarrow \textsc{FitEncoder}(\pi_t(\mathcal{D}_{\text{tr}} \setminus \mathcal{F}_j))$
      $g \leftarrow \textsc{Train}_m(\phi(\pi_t(\mathcal{D}_{\text{tr}} \setminus \mathcal{F}_j)), \theta)$
      $c_j \leftarrow F_1(g, \phi(\pi_t(\mathcal{F}_j)))$
    $\text{cv}(\theta) \leftarrow \frac{1}{5}\sum_j c_j$
  $\theta^*_{m,t} \leftarrow \arg\max_\theta \text{cv}(\theta)$ ▷ k-NN: extend if at edge
  $\phi \leftarrow \textsc{FitEncoder}(\pi_t(\mathcal{D}_{\text{tr}}))$
  $g \leftarrow \textsc{Train}_m(\phi(\pi_t(\mathcal{D}_{\text{tr}})), \theta^*_{m,t})$
  $\mathbf{q}_{m,t} \leftarrow \textsc{Metrics}(g, \phi(\pi_t(\mathcal{D}_{\text{te}})))$
procedure $\textsc{Train}_{\text{LR}}(X, \mathbf{y}, (\eta, T, |B|))$
  $\boldsymbol{\theta} \sim U(-0.01, 0.01)^{d+1}$
  for epoch $= 1..T$ **do**
    for each batch B of a seeded permutation of rows **do**
      $\hat{\mathbf{y}} \leftarrow \sigma([X_B, \mathbf{1}]\boldsymbol{\theta})$
      $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \frac{\eta}{|B|}[X_B, \mathbf{1}]^\top\big((1 - 2\mathbf{y}_B) \odot \hat{\mathbf{y}} \odot (1 - \hat{\mathbf{y}})\big)$
  return $\boldsymbol{\theta}$
$\textsc{Train}_{\text{KNN}}$ stores the encoded training rows and k.

each weight shows how one encoded attribute moves the log-odds of churn. Unlike cross-entropy, this loss is non-convex in $\boldsymbol{\theta}$, so gradient descent is not guaranteed to reach a global optimum, and $s(\mathbf{x})$ is a churn score rather than a calibrated probability; we therefore treat 0.5 as a decision threshold and report ranking metrics (ROC/PR-AUC) alongside F_1. The grid is $\eta \in \{0.01, 0.1, 0.5\}$, epochs $T \in \{50, 200\}$ and batch size $|B| \in \{32, 128, 512\}$ (18 settings).

2.5 Model Selection and Inference

For each model, tier and setting θ, the CV score is the mean churn $F_1 = \frac{2PR}{P+R}$ over the five folds, each fold trained on $\mathcal{D}_{\text{tr}} \setminus \mathcal{F}_j$ and scored on $\mathcal{F}_j$; $\theta^* = \arg\max_\theta$ of that score. F-measure is used because churners are the minority class and both missed churners and false alarms matter. The model is then refit on all of $\mathcal{D}_{\text{tr}}$ with θ^* and evaluated *once* on $\mathcal{D}_{\text{te}}$, so $\mathcal{D}_{\text{te}}$ never influences any choice.

3 Pseudo-code and Illustration

Algorithm 1 gives the complete procedure, including the LR training routine, and Figure 1 illustrates the data flow. The tier loop is the experimental variable; everything inside it is identical across tiers, so differences in $\mathbf{q}_{m,t}$ can be attributed to the information available rather than to the protocol.

4 Implementation

The implementation follows the FAIR principles and the course's recommended structure. `src/main.py` runs Algorithm 1 end to end. Shared steps live in `src/cmn/`: `tiers.py` ($\mathcal{A}_t$), `data.py` (Clean, split, folds, ϕ_t), `metrics.py` and `config.py` (seed, folds, paths).

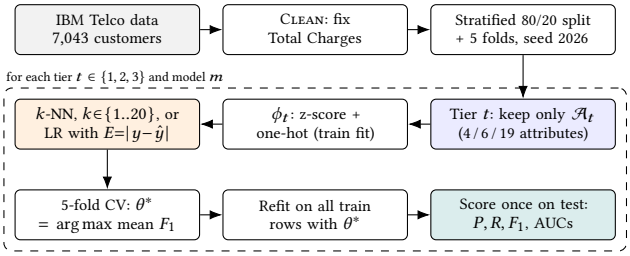


Figure 1: PRISM pipeline. Every tier and model share the same cleaned data, split and folds; a model only sees the attributes of its tier.

Table 1: Selected settings and hold-out results from the current implementation (sanity run; full evaluation is Milestone 4).

Tier	Model	θ^*	Test F_1	ROC-AUC
1	k-NN	$k = 15$	0.487	0.781
	LR	η=0.5, T=200, $\lvert B\rvert$=32	0.538	0.782
2	k-NN	$k = 13$	0.580	0.815
	LR	η=0.5, T=200, $\lvert B\rvert$=32	0.564	0.807
3	k-NN	$k = 19$	0.623	0.838
	LR	η=0.5, T=50, $\lvert B\rvert$=32	0.621	0.842

Each model has its own package: `src/knn/` for the k sweep and `src/logistic_regression/` for $\textsc{Train}_{\text{LR}}$, written directly in NumPy; `src/compare.py` writes the side-by-side table and chart. Results go to `output/<model>/` as CSV/JSON, and charts and analysis notebooks to `evaluation/`. A single `quickstart.sh` creates a virtual environment, installs the pinned `requirements.txt`, restores the dataset if missing, and runs the pipeline (under two minutes;`-fast` runs only k-NN). All randomness comes from seed 2026, and each run records library versions in `output/run_config.json`, so the same inputs and settings produce the same outputs.

References

[1] Thomas M. Cover and Peter E. Hart. 1967. Nearest Neighbor Pattern Classification. *IEEE Transactions on Information Theory* 13, 1 (1967), 21–27. doi:10.1109/TIT.1967.1053964

[2] Abigail Goldsteen, Gilad Ezov, Ron Shmelkin, Micha Moffie, and Ariel Farkash. 2022. Data Minimization for GDPR Compliance in Machine Learning Models. *AI and Ethics* 2, 3 (2022), 477–491. doi:10.1007/s43681-021-00095-8

[3] IBM. 2019. Telco Customer Churn. IBM Cognos Analytics sample data set (7,043 customers, 33 attributes).

[4] Michael C. Mozer, Richard Wolniewicz, David B. Grimes, Eric Johnson, and Howard Kaushansky. 2000. Predicting Subscriber Dissatisfaction and Improving Retention in the Wireless Telecommunications Industry. *IEEE Transactions on Neural Networks* 11, 3 (2000), 690–696. doi:10.1109/72.846740

[5] Scott A. Neslin, Sunil Gupta, Wagner Kamakura, Junxiang Lu, and Charlotte H. Mason. 2006. Defection Detection: Measuring and Understanding the Predictive Accuracy of Customer Churn Models. *Journal of Marketing Research* 43, 2 (2006), 204–211. doi:10.1509/jmkr.43.2.204

[6] Asem Omari, Omaia Al-Omari, Tariq Al-Omari, and Suliman Mohamed Fati. 2025. A Predictive Analytics Approach to Improve Telecom's Customer Retention. *Frontiers in Artificial Intelligence* 8 (2025), 1600357. doi:10.3389/frai.2025.1600357

A AI Task Completion Disclosure (Category D)

AI use. Claude (Anthropic) co-wrote the first LR experiment, merged the k-NN and LR work into one pipeline with shared code, a comparison step, README and quickstart, and drafted this report, including this appendix. OpenAI Codex fixed LR to the absolute-error loss, regenerated its results and renamed the loss. No AI use is recorded for the k-NN baseline. The team chose the design (k-NN baseline vs. LR, both taught in class) and kept the course loss function.

A.1 Contributions and Evaluation of AI Output

Drafted by Claude from the repository history; each member confirms their own statement.

Abhishek Majumdar set up the project and implemented the k-NN baseline: cleaning, 5-fold CV k sweep, test evaluation and analysis plots. The AI restructure kept his cleaning rules, tiers and k selection; run in the same environment, his original code and the restructured code give identical results.

Dev Parsana added instructor and TA access to the repository, and directed the AI-assisted integration of the k-NN and LR work into one pipeline, the shared-code restructure, the reproducibility rerun, the README and quickstart, and this report. He had the work audited against the manual, which found that `main` was outdated; the team overruled the AI's suggestion to drop the course loss.

Ravi Vashi added the earlier milestone files and implemented the LR experiment with grid search, later fixing it to the course loss. A tier document produced with AI stated the wrong seed and fill rule and was corrected; his updated LR results reproduce exactly with the pinned dependencies.